ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN – TIN HỌC



ĐỒ ÁN THỰC HÀNH GIỮA KÌ

NHẬN DẠNG

NHẬN DẠNG CHỮ SỐ VIẾT TAY MNIST BẰNG CÁC PHƯƠNG PHÁP PHÂN LỚP

Ngành: Toán tin - KHDL

Bộ môn: Ứng dụng Tin học

Giảng viên: TS. Trương Phước Hưng
CN. Lê Phú Trường

Sinh viên: Lê Huy **MSSV:** 24110022
Trương Hoàng Phúc **MSSV:** 24280020
Trần Phương Thảo **MSSV:** 24110242

Thành phố Hồ Chí Minh, ngày 8 tháng 11 năm 2026

Tóm tắt

Báo cáo so sánh bốn mô hình phân loại chữ số viết tay MNIST: Multinomial Naive Bayes, Softmax Regression, MLP và Ensemble (Random Forest). Dữ liệu gồm 50.000 mẫu huấn luyện, 10.000 mẫu validation và 10.000 mẫu kiểm thử, được tách từ bộ dữ liệu MNIST gốc với seed 42. Ba mô hình đầu được cài đặt thủ công, trong khi Random Forest sử dụng thuật toán của scikit-learn. Siêu tham số được lựa chọn trên tập validation và đánh giá trên tập test.

Kết quả cho thấy MLP đạt accuracy cao nhất (98,47%, macro-F1 98,46%), tiếp theo là Random Forest (96,10%), Softmax Regression (92,22%) và Naive Bayes (83,61%). Về thời gian huấn luyện, Naive Bayes nhanh nhất (0,06 giây), còn MLP chậm nhất (307 giây). Các mô hình tự cài đặt có accuracy chênh lệch tối đa 0,19 điểm phần trăm so với mô hình scikit-learn tương ứng.

Do chỉ thực nghiệm với một seed, kết quả chưa phản ánh đầy đủ tính ổn định của các mô hình.

1 Giới thiệu

MNIST là bộ dữ liệu chuẩn gồm 70.000 ảnh xám 28×28 của chữ số viết tay từ 0 đến 9 (60.000 ảnh huấn luyện và 10.000 ảnh kiểm thử). Do kích thước nhỏ và nhãn sạch, MNIST thuận lợi để so sánh các họ mô hình phân lớp trong cùng một quy trình thực nghiệm.

Báo cáo này trình bày Đồ án 1 (giữa kỳ) của môn Nhận dạng: xây dựng và so sánh bốn mô hình: Multinomial Naive Bayes, Softmax Regression, MLP và Random Forest. Các mô hình được đánh giá qua accuracy, precision, recall, F1-score, thời gian thực thi và khả năng giải thích. Nhóm cũng khảo sát siêu tham số và đối chiếu kết quả với scikit-learn. Các câu hỏi mà báo cáo trả lời là:

1. Mỗi mô hình đạt độ chính xác và các độ đo macro (precision, recall, F1) bao nhiêu trên tập huấn luyện, validation và kiểm thử?
2. Các siêu tham số chính (smoothing, learning rate, điều chuẩn, kiến trúc, dropout, số cây, …) ảnh hưởng thế nào đến kết quả trên validation?
3. Các mô hình thường nhầm những cặp chữ số nào, và đánh đổi giữa độ chính xác, thời gian huấn luyện, thời gian dự đoán và khả năng giải thích ra sao?
4. Cài đặt của nhóm có khớp với các mô hình tương ứng của scikit-learn [1] không?

Nhóm vận dụng kiến thức và quy trình thực hành từ bộ bài lab của học phần Nhận dạng [2], bao gồm đánh giá phân lớp, Naive Bayes, bộ phân lớp tuyến tính, MLP trên MNIST và phương pháp tổ hợp.

Cơ sở lý thuyết được đối chiếu với các tài liệu xuất hiện trong danh mục tham khảo của bài lab, gồm [1, 3, 4].

Quy trình thực nghiệm, cấu hình mô hình và các kết quả được trình bày dựa trên mã nguồn và Notebook đã chạy của nhóm.

Phạm vi thực hiện. Nhóm tự cài đặt Naive Bayes, Softmax Regression và MLP. Với Random Forest, nhóm tự triển khai bootstrap và biểu quyết đa số, sử dụng cây quyết định của scikit-learn. Các mô hình được kiểm thử và đánh giá trên MNIST.

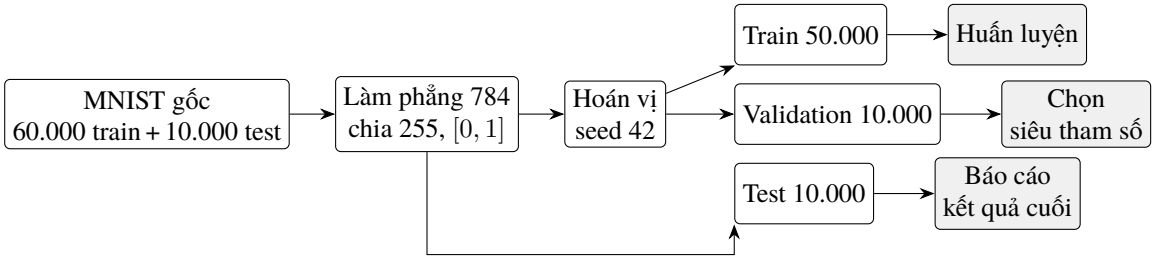
Bố cục. Mục 2 mô tả dữ liệu; Mục 3 trình bày công thức và thuật toán; Mục 4 nêu thiết kế thực nghiệm; Mục 5 báo cáo kết quả; Mục 6 tổng hợp so sánh và thảo luận; Mục 7 kết luận.

2 Dữ liệu và tiền xử lý

2.1 Nguồn dữ liệu và chia tập

Dữ liệu được tải bằng `torchvision.datasets.MNIST`.

- **Tiền xử lý:** mỗi ảnh xám 28×28 được làm phẳng thành vectơ $\boldsymbol{x} \in \mathbb{R}^{784}$, chuyển sang `float32` và chia cho 255 nên mọi pixel nằm trong $[0, 1]$; không dùng thêm phép chuẩn hoá z-score.
- **Chia tập:** 60.000 ảnh huấn luyện gốc được hoán vị ngẫu nhiên bằng bộ sinh số `default_rng` của NumPy với seed 42; 50.000 ảnh đầu dùng để huấn luyện và 10.000 ảnh còn lại làm validation.
- **Tập test:** tập test gốc (10.000 ảnh) được giữ riêng và không tham gia chọn siêu tham số (Hình 1).
- Phép chia là hoán vị ngẫu nhiên thông thường, không phân tầng theo nhãn.

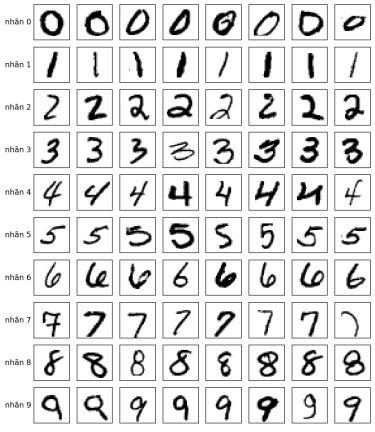


Hình 1. Quy trình dữ liệu: train để học tham số, validation để chọn siêu tham số, test chỉ dùng để báo cáo cuối.

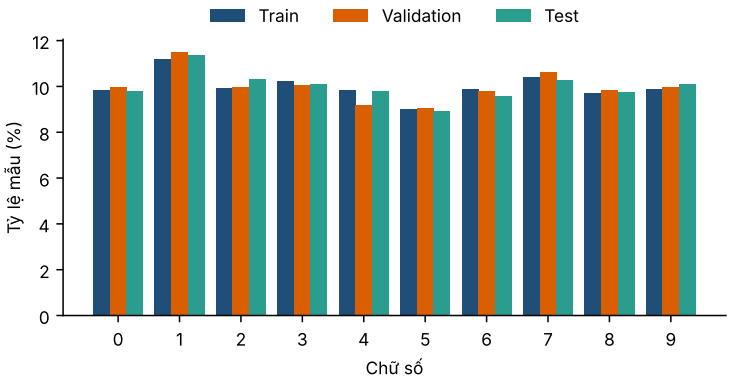
2.2 Khảo sát dữ liệu

Các tập có 784 đặc trưng, nhãn 10 lớp và giá trị pixel trong $[0, 1]$.

- Tỷ lệ các lớp xấp xỉ đều: trên tập train, lớp 1 chiếm nhiều nhất (11,19%, 5.594 ảnh) và lớp 5 ít nhất (9,03%, 4.516 ảnh); tỷ lệ trên validation và test có cùng xu hướng (Hình 2b).
- Do đó accuracy là độ đo hợp lý, và nhóm vẫn báo cáo thêm các độ đo macro để phản ánh chất lượng từng lớp.
- Hình 2a cho thấy mỗi chữ số có nhiều kiểu nét (độ nghiêng, độ đậm, cách viết nét), nên các mô hình phải chịu được biến thiên hình dạng trong cùng một lớp.



(a) Ảnh mẫu từng chữ số (train)



(b) Tỷ lệ mỗi lớp trong từng tập

Hình 2. Ảnh minh họa và phân bố nhãn của ba tập dữ liệu. Hình và số liệu phân bố nhãn lấy từ [5, 6] (`eda_class_distribution.csv`).

Khoảng cách Euclid giữa ảnh trung bình của các lớp cho thấy ba cặp gần nhau nhất là 4–9 (2,48), 7–9 (2,99) và 5–8 (3,10). Thông tin này được đối chiếu với các cặp bị nhầm thực tế ở Mục 5.3.

2.3 Lưu ý với Multinomial Naive Bayes

Multinomial Naive Bayes vốn dành cho dữ liệu đếm. Ở đây cường độ pixel $x_j \in [0, 1]$ được dùng như trọng số không âm (pseudo-count) trong tổng $\sum_j x_j \log \theta_{cj}$; đây là một xấp xỉ thực dụng, không phải mô hình đếm đúng nghĩa và không coi pixel là biến Bernoulli.

3 Phương pháp và mức độ tự cài đặt

Ký hiệu: tập huấn luyện $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$ với $\boldsymbol{x}_i \in [0,1]^{784}$, $y_i \in \{0, \ldots, 9\}$, $K = 10$ lớp, $d = 784$. Bảng 1 tóm tắt phần nhóm tự viết và phần dùng thư viện.

Bảng 1. Mức độ tự cài đặt của bốn mô hình.

Mô hình	Nhóm tự cài đặt	Thư viện hỗ trợ
Naive Bayes	prior, tổng cường độ theo lớp, smoothing, log-score, argmax (NumPy)	—
Softmax Regression	softmax ổn định, cross-entropy, gradient, mini-batch SGD, L2 (NumPy)	—
MLP	lan truyền xuôi/ngược, ReLU, inverted dropout, SGD + momentum, weight decay, early stopping	PyTorch: tensor, `nn.Linear` (lưu tham số); không dùng `loss.backward()`
Random Forest	bootstrap, vòng lặp tập cây, biểu quyết đa số	`DecisionTreeClassifier` của scikit-learn làm cây cơ sở

3.1 Multinomial Naive Bayes

Bài thực hành Tuần 2 cung cấp quy trình khám phá dữ liệu và kiểm chứng Multinomial Naive Bayes trên ảnh chữ số; cơ sở mô hình xác suất được đối chiếu với [4]. Gọi N_c là số ảnh lớp c và $s_{cj} = \sum_{i:y_i=c} x_{ij}$ là tổng cường độ của pixel j trong lớp c. Với hệ số làm mịn Laplace $\alpha > 0$:

$$P(y=c) = \frac{N_c}{N}, \qquad \theta_{cj} = \frac{s_{cj} + \alpha}{\sum_{k=1}^{d} s_{ck} + d\,\alpha}, \tag{1}$$

$$\text{score}_c(\boldsymbol{x}) = \log P(y=c) + \sum_{j=1}^{d} x_j \log \theta_{cj}, \qquad \hat{y} = \arg\max_{c}\ \text{score}_c(\boldsymbol{x}). \tag{2}$$

Mô hình giả định các pixel độc lập có điều kiện theo lớp; việc tính trong không gian log tránh tràn số dưới.

Thuật toán 1 Multinomial Naive Bayes

Đầu vào: X_{train}, y_{train}, α

1: **với** mỗi lớp c **thực hiện**

2: $\quad \pi_c \leftarrow N_c/N;\ s_c \leftarrow \sum_{i:y_i=c} \boldsymbol{x}_i;\ \theta_c \leftarrow (s_c + \alpha)/\sum_j (s_{cj} + \alpha)$

3: **kết thúc với**

4: **dự đoán:** $\hat{y}(\boldsymbol{x}) = \arg\max_c [\log \pi_c + \boldsymbol{x}^\top \log \theta_c]$

3.2 Softmax Regression

Bài thực hành Tuần 3 giới thiệu phân lớp tuyến tính và tối ưu gradient; nhóm mở rộng thành Softmax Regression đa lớp trên MNIST. Công thức hàm mất mát và gradient được đối chiếu với [4]. Với lô B

mẫu, ma trận đặc trưng $X_B \in \mathbb{R}^{B\times d}$, nhãn one-hot Y_B, trọng số $W \in \mathbb{R}^{d\times K}$ và độ chệch $\boldsymbol{b} \in \mathbb{R}^K$:

$$Z = X_B W + \mathbf{1}\boldsymbol{b}^\top, \qquad p_{ik} = \frac{\exp(z_{ik} - \max_l z_{il})}{\sum_l \exp(z_{il} - \max_{l'} z_{il'})}, \tag{3}$$

$$L = -\frac{1}{B}\sum_i \log p_{i,y_i} + \frac{\lambda}{2}\|W\|_F^2, \tag{4}$$

$$\nabla_W L = \frac{1}{B} X_B^\top (P - Y_B) + \lambda W, \qquad \nabla_{\boldsymbol{b}} L = \frac{1}{B}\sum_i (\boldsymbol{p}_i - \boldsymbol{y}_i). \tag{5}$$

Việc trừ $\max_l z_{il}$ trong (3) giữ ổn định số học. Tham số được cập nhật bằng mini-batch SGD $W \leftarrow W - \eta\nabla_W L$, $\boldsymbol{b} \leftarrow \boldsymbol{b} - \eta\nabla_{\boldsymbol{b}} L$; độ chệch không bị điều chuẩn. Khởi tạo bằng 0, 20 epoch, kích thước lô 128.

Thuật toán 2 Softmax Regression (mini-batch SGD)

Đầu vào: X, y, η, λ, số epoch T, kích thước lô B
1: $W \leftarrow 0$, $\boldsymbol{b} \leftarrow 0$
2: **với** epoch $= 1, \ldots, T$ **thực hiện**
3: hoán vị chỉ số mẫu
4: **với** mỗi lô **thực hiện**
5: tính Z, P theo (3); tính $\nabla_W L$, $\nabla_{\boldsymbol{b}} L$ theo (5); cập nhật $W, \boldsymbol{b}$
6: **kết thúc với**
7: **kết thúc với**
8: **dự đoán:** $\hat{y} = \arg\max_k (\boldsymbol{x}^\top W + \boldsymbol{b})_k$

3.3 Mạng nơ-ron nhiều lớp (MLP)

Lab Tuần 4 hướng dẫn MLP trên MNIST, lan truyền ngược thủ công và khảo sát siêu tham số; nhóm vận dụng và mở rộng trong Notebook. Nền tảng mạng nơ-ron được đối chiếu với [3, 4]. Mạng gồm các lớp tuyến tính xen ReLU (và dropout) rồi một lớp đầu ra 10 logits (Hình 3). Với $A^{(0)} = X_B$:

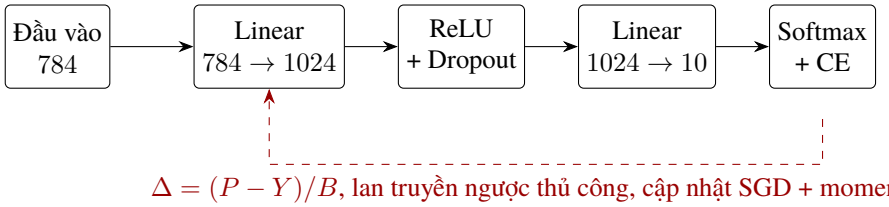
$$Z^{(l)} = A^{(l-1)} {W^{(l)}}^\top + \mathbf{1}{\boldsymbol{b}^{(l)}}^\top, \qquad A^{(l)} = \max(0, Z^{(l)}) \text{ (tầng ẩn)}, \tag{6}$$

Hàm mất mát là cross-entropy trung bình trên softmax của logits đầu ra. Lan truyền ngược thực hiện thủ công: sai số tại đầu ra $\Delta = (P - Y_B)/B$; với mỗi lớp tuyến tính

$$\nabla_{W^{(l)}} = \Delta^\top A^{(l-1)} + \lambda W^{(l)}, \qquad \nabla_{\boldsymbol{b}^{(l)}} = \textstyle\sum_i \Delta_i, \qquad \Delta_{\text{in}} = \Delta W^{(l)}, \tag{7}$$

Qua ReLU, sai số được nhân với chỉ thị $\mathbf{1}[Z^{(l)} > 0]$ (nhân thêm mặt nạ dropout đã chia $1-p$ nếu có). Các thành phần huấn luyện:

- **SGD với momentum:** $v \leftarrow \mu v - \eta\nabla$, $w \leftarrow w + v$.
- **Inverted dropout:** nhân mặt nạ Bernoulli rồi chia $1-p$ khi huấn luyện; tắt khi suy luận.
- **Khởi tạo:** trọng số He (Kaiming), độ chệch bằng 0.
- **Early stopping** theo cross-entropy trên validation: huấn luyện dừng sau P epoch liên tiếp không cải thiện và trọng số tốt nhất được khôi phục.
- **Weight decay:** khi $\lambda > 0$, train loss trong lịch sử huấn luyện gồm cả số hạng L2 còn validation loss thì không; vì vậy hai đường này không được so trực tiếp ở các thí nghiệm có weight decay.



Hình 3. MLP cấu hình cuối (784–1024–10): đường liền là lan truyền xuôi, nét đứt là lan truyền ngược.

Thuật toán 3 Huấn luyện MLP

Đầu vào: X, y, X_{val}, y_{val}, kiến trúc, η, μ, p, λ, $T_{\max}$, kiên nhẫn P

với epoch $= 1, \ldots, T_{\max}$ **thực hiện**
  với mỗi lô **thực hiện**
    lan truyền xuôi, lưu $A^{(l)}$ và $Z^{(l)}$; tính cross-entropy và Δ
    lan truyền ngược theo (7); cập nhật v, w bằng SGD + momentum
  kết thúc với
  đánh giá train/validation (dropout tắt); nếu validation loss giảm thì lưu trọng số, ngược lại tăng bộ đếm
  nếu bộ đếm $\geq P$ **thì**
    dừng
  kết thúc nếu
kết thúc với
khôi phục trọng số tốt nhất

3.4 Random Forest

Bài thực hành Tuần 5 giới thiệu các phương pháp ensemble và cách so sánh với cây đơn; nhóm vận dụng ý tưởng đó vào khảo sát số cây trên MNIST. Rừng gồm T cây. Với cây thứ t, nhóm:

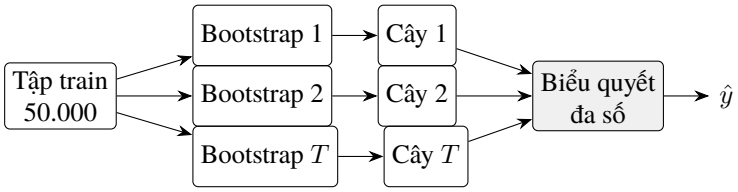
- lấy N chỉ số có hoàn lại từ tập train (bootstrap);
- huấn luyện một `DecisionTreeClassifier` (độ sâu tối đa 15, tại mỗi nút xét $\lfloor\sqrt{784}\rfloor = 28$ đặc trưng ngẫu nhiên, `random_state`$= 42 + t$).

Dự đoán bằng biểu quyết đa số cứng, đồng hạng chọn nhãn nhỏ nhất:

$$\hat{y}(\boldsymbol{x}) = \arg\max_{c} \sum_{t=1}^{T} \mathbf{1}[h_t(\boldsymbol{x}) = c]. \tag{8}$$

Nhóm không tự viết thuật toán chia nút (Gini) của cây quyết định; phần tự cài đặt chỉ gồm bootstrap, vòng lặp tập cây và biểu quyết.

Việc dùng cây cơ sở của scikit-learn là phương án đã được giảng viên chấp thuận; nhóm vẫn phân biệt rõ phần do nhóm triển khai và phần do thư viện đảm nhiệm.



Hình 4. Random Forest: $T = 50$ cây huấn luyện trên các mẫu bootstrap rồi biểu quyết.

Thuật toán 4 Random Forest (mức triển khai thực tế)

Đầu vào: X, y, T, độ sâu tối đa, `max_features`

1: **với** $t = 1, \ldots, T$ **thực hiện**
2: lấy N chỉ số có hoàn lại; huấn luyện `DecisionTreeClassifier` (scikit-learn) trên mẫu bootstrap
3: **kết thúc với**
4: **dự đoán:** mỗi cây dự đoán nhãn; chọn nhãn có số phiếu lớn nhất theo (8)

3.5 Các độ đo

Cách tính các độ đo và đọc ma trận nhầm lẫn được vận dụng từ bài thực hành Tuần 1 và kiểm chứng với scikit-learn [1]. Từ ma trận nhầm lẫn C (hàng là nhãn thật, cột là nhãn dự đoán), với lớp k: $\text{TP}_k = C_{kk}$, $\text{FP}_k = \sum_{i \neq k} C_{ik}$, $\text{FN}_k = \sum_{j \neq k} C_{kj}$,

$$P_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad R_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \quad F1_k = \frac{2P_k R_k}{P_k + R_k}, \quad \text{Acc} = \frac{\sum_k C_{kk}}{N}, \tag{9}$$

và các độ đo macro là trung bình số học trên 10 lớp (mẫu số bằng 0 cho giá trị 0).

4 Thiết kế thực nghiệm và kiểm chứng

4.1 Môi trường và quy trình

Thực nghiệm chạy trên Google Colab (CPU) với Python 3.13.18, NumPy 2.4.6, PyTorch 2.11.0 (bản CPU) và scikit-learn 1.6.1; seed $= 42$ cho chia dữ liệu, khởi tạo và hoán vị. Mô hình MLP chạy trên CPU. Quy trình gồm:

(i) chạy baseline;
(ii) khảo sát từng siêu tham số trên validation;
(iii) chọn cấu hình theo validation accuracy;
(iv) huấn luyện lại cấu hình cuối trên tập train;
(v) đánh giá accuracy, precision/recall/F1 macro và ma trận nhầm lẫn trên train, validation và test;
(vi) so sánh với scikit-learn.

Tập test không tham gia bước (ii)–(iii). Với MLP, validation còn dùng cho early stopping; khi chọn epoch tốt nhất theo validation loss, accuracy validation báo cáo là của trọng số được khôi phục.

4.2 Các khảo sát siêu tham số

Bảng 2 liệt kê các khoảng giá trị đã thử.

- Mỗi khảo sát thay đổi một nhóm tham số trong khi giữ phần còn lại ở giá trị baseline.
- Cấu hình cuối của MLP ghép các giá trị tốt nhất từ từng khảo sát rồi huấn luyện lại tối đa 30 epoch với `patience`$= 5$.
- Cách ghép này không phải tìm kiếm lưới đồng thời, nên tổ hợp cuối chưa được kiểm tra riêng lẻ như một điểm của lưới.

Bảng 2. Các siêu tham số được khảo sát và cấu hình cuối. Các khảo sát MLP có số epoch khác nhau; early stopping dựa trên validation loss.

Mô hình	Tham số được khảo sát	Cấu hình baseline	Giá trị được chọn
Naive Bayes	$\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$	$\alpha = 0{,}01$	$\alpha = 10^{-3}$
Softmax	$\eta \in \{0{,}01; 0{,}05; 0{,}1; 0{,}5\}$; $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$	$\eta = 0{,}1$, $\lambda = 10^{-3}$, 20 epoch, lô 128	$\eta = 0{,}1$, $\lambda = 0$
MLP	$\eta \in \{0{,}005; 0{,}01; 0{,}05; 0{,}1; 0{,}5\} \times \mu \in \{0; 0{,}5; 0{,}9\}$; lô $\{16, 64, 256, 1024\}$; lớp ẩn $\{[64], [256], [1024], [256, 128], 4\times256\}$; dropout $\{0; 0{,}2; 0{,}5\}$; weight decay $\{0, 10^{-4}, 10^{-3}\}$	$[256, 128]$, $\eta = 0{,}05$, $\mu = 0{,}9$, lô 128, 10 epoch	$[1024]$, $\eta = 0{,}1$, $\mu = 0{,}9$, lô 64, $p = 0{,}2$, $\lambda = 0$
Random Forest	$T \in \{10, 30, 50\}$	$T = 50$, độ sâu 15, $\sqrt{d}$ đặc trưng	$T = 50$

- Các khảo sát learning rate×momentum, kích thước lô và kiến trúc MLP chạy 5 epoch mỗi cấu hình.
- Khảo sát dropout chạy 20 epoch trên kiến trúc baseline $[256, 128]$.
- Vì điều kiện và số epoch khác nhau, *không so sánh trực tiếp* validation accuracy giữa các khảo sát.
- Với cấu hình cuối, validation loss dùng để chọn epoch tốt nhất thông qua early stopping.
- Đây là khảo sát siêu tham số trên một lần chia dữ liệu (seed 42), không phải đánh giá lặp nhiều seed; tập test chỉ được dùng sau khi chốt cấu hình.

4.3 Kiểm chứng cài đặt

- **Kiểm thử đơn vị:** bộ `pytest` gồm 12 trường hợp, bao gồm kiểm tra công thức Naive Bayes, gradient và độ ổn định số của Softmax Regression, gradient MLP, cập nhật momentum, dropout, early stopping và cơ chế biểu quyết của Random Forest. Cả 12 kiểm thử đều `PASSED`; chi tiết được lưu trong `results/unit_tests.csv` và `results/pytest_report.xml`.
- **Đối chiếu Naive Bayes** với `MultinomialNB` của scikit-learn: sai khác lớn nhất của log-prior và log-likelihood lần lượt là $8{,}9 \times 10^{-16}$ và $1{,}2 \times 10^{-5}$; dự đoán trùng khớp 100% trên tập validation.
- **Đối chiếu các độ đo:** accuracy, precision, recall, F1 macro và ma trận nhầm lẫn được đối chiếu với `sklearn.metrics` trên 12 tổ hợp mô hình và tập dữ liệu, với sai khác lớn nhất $2{,}2 \times 10^{-16}$.
- Kết quả đối chiếu nằm trong các tệp `nb_sklearn_check.csv` và `final_metrics_sklearn_check.csv`.

4.4 Đo thời gian

- Thời gian được đo bằng `time.perf_counter()` (giây).
- Thời gian huấn luyện là thời gian của lần `fit` gốc được lưu cùng cache, không phải thời gian đọc lại cache.
- Với MLP, thời gian này gồm cả việc đánh giá train/validation sau mỗi epoch ($265{,}5$ s huấn luyện và $41{,}5$ s đánh giá trong tổng $307{,}0$ s).
- Các phép đo chạy trên máy ảo Colab dùng chung nên chỉ mang tính so sánh tương đối; cấu hình CPU cụ thể không được ghi lại.

Nguồn tài liệu và dữ liệu thực nghiệm. Phương pháp và quy trình thực hành tham khảo bộ bài lab của học phần [2]. Toàn bộ kết quả của nhóm được công bố tại ba nơi:

- bảng số liệu (CSV, JSON, NPZ): [6];
- hình ảnh và biểu đồ: [5];
- mã nguồn và Notebook: [7].

Mọi bảng và hình trong báo cáo được lập từ các tệp này; tên tệp nguồn được ghi trong chú thích khi cần truy xuất.

5 Kết quả và phân tích

Các bảng trong mục này lập từ tệp kết quả [6]; các hình được vẽ từ cùng nguồn hoặc lấy từ hình xuất của Notebook [5].

5.1 Kết quả tổng quát

Bảng 3 trình bày bốn độ đo của bốn mô hình ở cấu hình cuối trên ba tập.

- MLP đạt kết quả cao nhất ở mọi tập validation và test: accuracy test $98{,}47\%$ và macro-F1 $98{,}46\%$.
- Accuracy test của MLP cao hơn Random Forest $2{,}37$ điểm phần trăm (pp), Softmax Regression $6{,}25$ pp và Naive Bayes $14{,}86$ pp.
- Với một tập test gồm 10.000 ảnh, các chênh lệch này phản ánh hiệu năng trong thiết lập đánh giá này.
- Nhóm chưa đánh giá lặp nhiều seed hoặc kiểm định ghép cặp giữa các mô hình, nên không suy diễn ý nghĩa thống kê từ riêng độ lớn chênh lệch.

Bảng 3. Accuracy và các độ đo macro (%) của bốn mô hình trên ba tập (nguồn: `final_metrics.csv`).

Mô hình	Tập	Accuracy	Precision	Recall	F1
Naive Bayes	Train	82,43	83,15	82,14	82,31
	Validation	82,57	83,17	82,20	82,37
	Test	**83,61**	**84,30**	**83,30**	**83,46**
Softmax Regression	Train	92,41	92,34	92,30	92,31
	Validation	91,87	91,79	91,74	91,74
	Test	**92,22**	**92,14**	**92,11**	**92,10**
MLP	Train	99,98	99,98	99,98	99,98
	Validation	98,32	98,32	98,29	98,31
	Test	**98,47**	**98,48**	**98,45**	**98,46**
Random Forest	Train	99,67	99,68	99,66	99,67
	Validation	95,94	95,90	95,90	95,90
	Test	**96,10**	**96,08**	**96,06**	**96,07**

Chênh lệch accuracy train–test (Bảng 5):

- MLP: $1{,}51$ pp (accuracy train $99{,}98\%$).
- Random Forest: $3{,}57$ pp (accuracy train $99{,}67\%$).
- Softmax Regression: $0{,}19$ pp.
- Naive Bayes: $83{,}61\%$ trên test so với $82{,}43\%$ trên train. Chênh lệch âm không đồng nghĩa với việc mô hình tổng quát hóa tốt hơn.

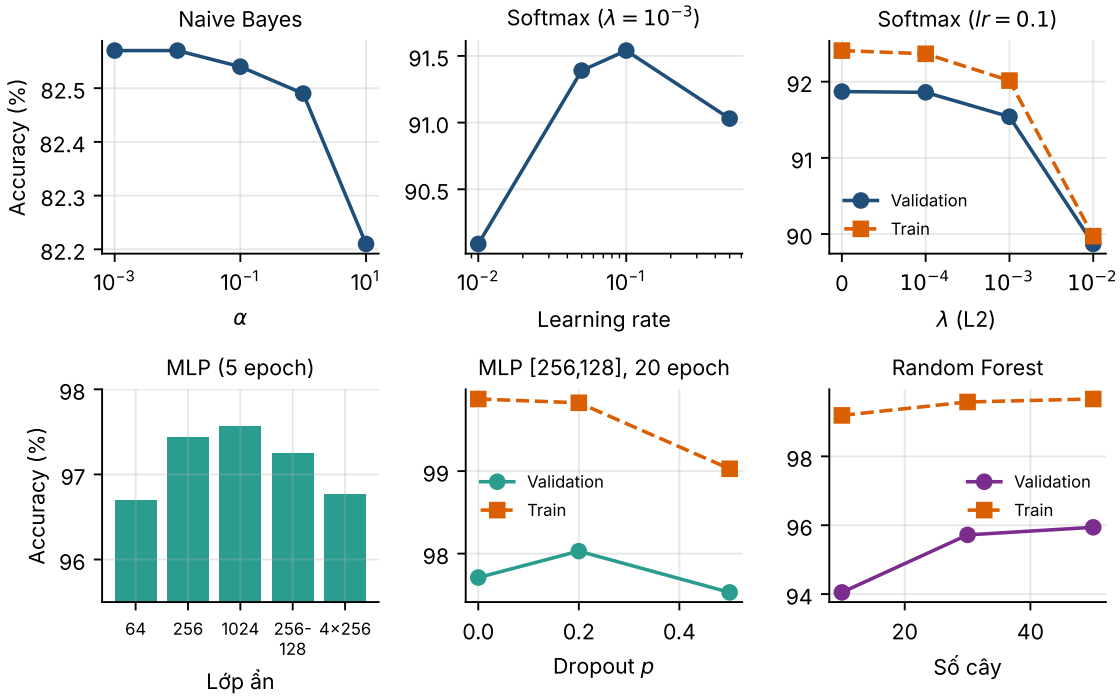
Chênh lệch ở MLP và Random Forest là dấu hiệu cần lưu ý về khả năng tổng quát hóa, đặc biệt đối với Random Forest. Một phép chia dữ liệu chưa đủ để xác định nguyên nhân.

5.2 Khảo sát siêu tham số và lựa chọn cấu hình

Hình 5 tổng hợp sáu khảo sát, trình bày theo hai hàng để dễ đối chiếu từng mô hình.

- **Naive Bayes:** validation accuracy chỉ thay đổi từ $82{,}21\%$ ($\alpha = 10$) đến $82{,}57\%$ ($\alpha \leq 0{,}01$), nghĩa là mô hình rất ít nhạy với smoothing. Có thể vì với 50.000 ảnh, các tổng s_{cj} lớn hơn nhiều so với α. $\alpha = 10^{-3}$ và 10^{-2} bằng nhau ở bốn chữ số thập phân nên nhóm chọn giá trị nhỏ hơn.
- **Softmax (learning rate):** $\eta = 0{,}1$ cho validation tốt nhất ($91{,}54\%$ khi $\lambda = 10^{-3}$); $\eta = 0{,}01$ cho $90{,}09\%$ với loss cuối còn cao ($0{,}387$); $\eta = 0{,}5$ kém hơn ($91{,}03\%$).
- **Softmax (điều chuẩn):** khi tăng λ từ 0 lên 10^{-2}, khoảng cách train–validation giảm từ $0{,}54$ xuống

0,10 pp nhưng validation accuracy giảm từ 91,87% xuống 89,87% (underfitting), nên $\lambda = 0$ được chọn.



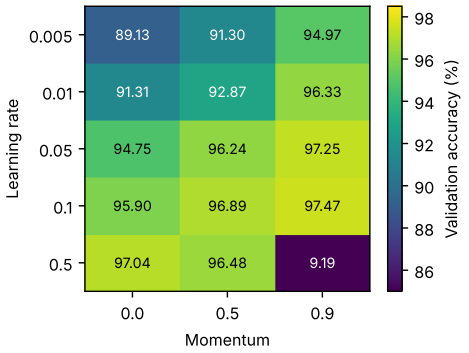
Hình 5. Sáu khảo sát siêu tham số: Naive Bayes và Softmax (hàng trên), kiến trúc MLP, dropout và số cây Random Forest (hàng dưới). Các đường train chỉ xuất hiện khi có dữ liệu tương ứng.

MLP.

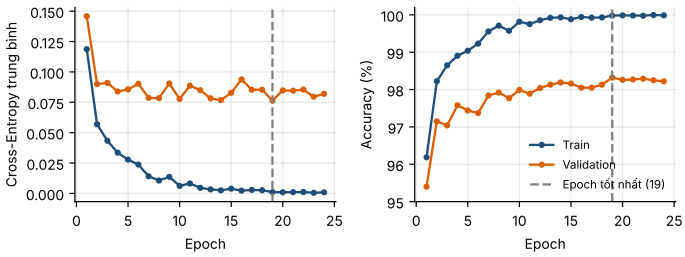
- **Learning rate và momentum** (Hình 6a, 5 epoch): validation accuracy thay đổi mạnh theo tổ hợp.
  - $\eta = 0{,}5,\ \mu = 0{,}9$ chỉ đạt 9,19% sau 5 epoch, gần mức dự đoán một lớp.
  - Tốt nhất là $\eta = 0{,}1,\ \mu = 0{,}9$ với 97,47%.
  - Tỷ số $\eta/(1-\mu)$ có thể dùng để tham khảo nhưng không đại diện cho một learning rate thực tế cố định: $\eta = 0{,}5,\ \mu = 0{,}5$ và $\eta = 0{,}1,\ \mu = 0{,}9$ cùng có tỷ số bằng 1 nhưng cho 96,48% và 97,47%.
- **Kích thước lô** (cùng 5 epoch):
  - Lô 64 và 256 cho 97,36% và 97,34%, trong khi lô 16 (95,17%) và lô 1024 (95,10%) kém hơn.
  - Lô 16 cần 62,6 s so với 16,4 s của lô 64 (15.625 so với 3.910 bước cập nhật).
  - Lô 1024 chỉ có 245 bước nên có thể chưa đủ để hội tụ trong 5 epoch.
- **Kiến trúc** (cùng 5 epoch): một lớp ẩn 1024 nơ-ron (814.090 tham số) cho 97,56%, hơn $[256]$ (97,44%), $[256, 128]$ (97,25%), 4×256 (96,76%) và $[64]$ (96,69%). Với chỉ 5 epoch, mạng sâu hơn không tốt hơn.
- **Dropout** (riêng trên kiến trúc baseline $[256, 128]$, 20 epoch): $p = 0{,}2$ đạt 98,03% validation, so với 97,71% khi $p = 0$; khoảng cách train–validation giảm từ 2,16 xuống 1,80 pp. Với $p = 0{,}5$, validation accuracy là 97,53%. Kết quả này không phải một phép ablation riêng trên kiến trúc MLP cuối $[1024]$.
- **Weight decay:** 10^{-4} và 10^{-3} không cải thiện accuracy (97,69% và 97,70% so với 97,72%) dù giảm khoảng cách train–validation xuống 1,35 pp ở 10^{-3}, nên không được chọn.
- **Early stopping** (`patience`$= 5$): dừng ở epoch 14 (tốt nhất tại epoch 9) và mất 36,4 s so với 79,6 s khi chạy đủ 30 epoch, cho cùng kết quả 97,71%. Chạy 30 epoch không dùng validation cho 98,02% accuracy nhưng cross-entropy validation lớn hơn (0,0900 so với 0,0767), vì tiêu chí dừng của nhóm là validation loss chứ không phải accuracy.

MLP cuối. Cấu hình 784–1024–10, $\eta = 0{,}1$, $\mu = 0{,}9$, lô 64, $p = 0{,}2$, $\lambda = 0$:

- Validation loss nhỏ nhất ở epoch 19, huấn luyện dừng ở epoch 24; validation accuracy là 98,32% (Hình 6b).



(a) Learning rate × momentum (5 epoch)



(b) Đường cong học của MLP cấu hình cuối

Hình 6. Khảo sát learning rate/momentum và quá trình huấn luyện của MLP cuối (nguồn: `mlp_lr_momentum.csv`, `mlp_final_history.csv`).

- Train loss giảm về gần 0, còn validation loss chỉ dao động trong khoảng 0,076–0,094 từ epoch 2 trở đi.
- Như vậy mạng khớp tập train gần hoàn toàn trong khi lợi ích trên validation đã bão hoà.

Random Forest.

- Tăng số cây từ 10 lên 30 và 50 nâng validation từ 94,05% lên 95,72% và 95,94%, đồng thời thời gian huấn luyện tăng từ 5,7 s lên 19,6 s và 34,9 s.
- Mức tăng từ 30 lên 50 cây chỉ 0,22 pp validation accuracy nhưng tốn thêm khoảng 15,3 s. Do $T = 50$ là biên lưới, chưa kết luận được xu hướng khi tăng thêm cây.
- Một cây đơn (độ sâu 15) chỉ đạt 87,38% validation, thấp hơn rừng 50 cây 8,56 pp; khoảng cách train–validation tương ứng là 11,07 pp so với 3,73 pp. Đây là bằng chứng trực tiếp về lợi ích của tổ hợp cây trong cùng thiết lập dữ liệu.

5.3 Ma trận nhầm lẫn và phân tích lỗi

Trên tập test, số mẫu bị phân loại sai là 1.639 (Naive Bayes), 778 (Softmax), 390 (Random Forest) và 153 (MLP) trên 10.000 (Hình 7). Bảng 4 liệt kê ba cặp bị nhầm nhiều nhất theo hai chiều.

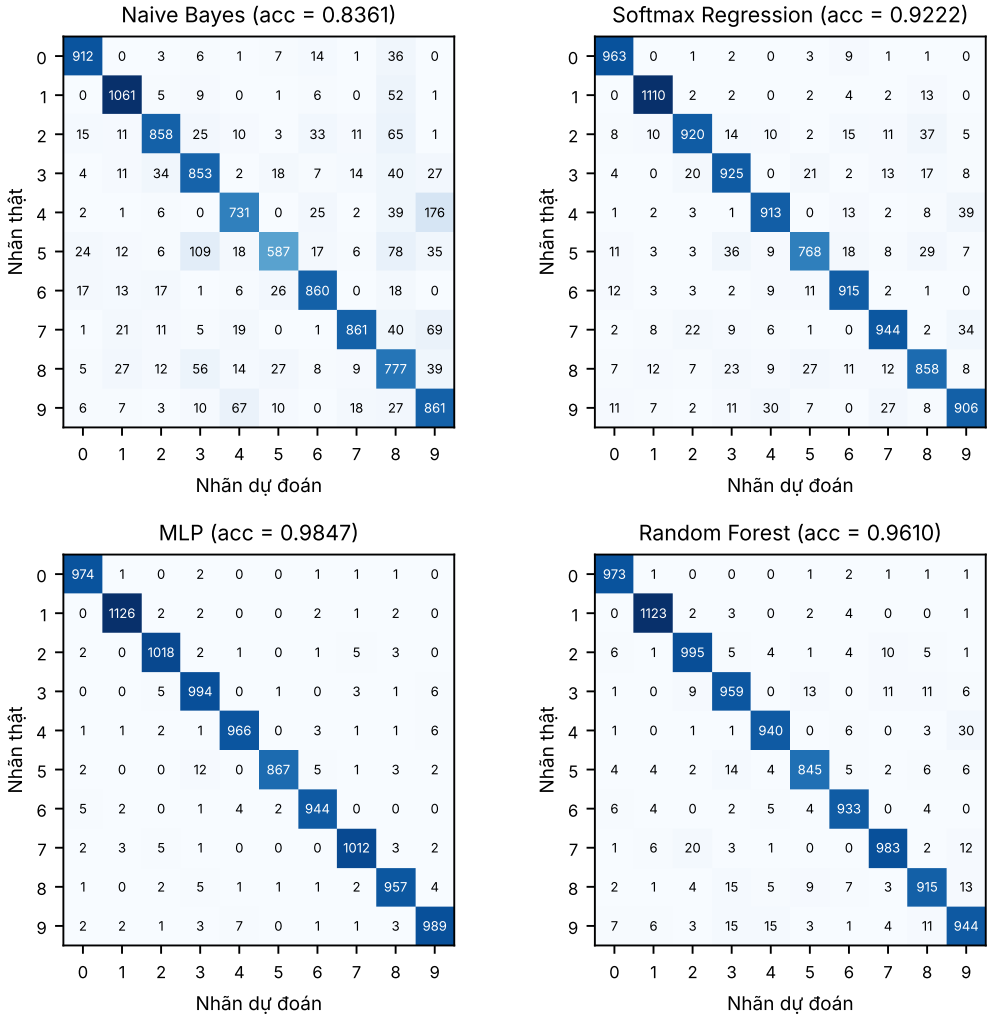
- **Cặp** $4 \leftrightarrow 9$ đứng đầu ở Naive Bayes (243 lỗi, trong đó 176 ảnh số 4 bị gán 9), Softmax (69) và Random Forest (45). Đây cũng là cặp có ảnh trung bình gần nhau nhất (Mục 2); các cặp $5 \leftrightarrow 8$ (Naive Bayes) và $7 \leftrightarrow 9$ (Softmax) cũng nằm trong ba cặp gần nhất.
- **Cặp** $3 \leftrightarrow 5$ **và** $2 \leftrightarrow 7$**:** $3 \leftrightarrow 5$ có 127 lỗi ở Naive Bayes (trong đó 109 ảnh số 5 bị gán 3). Hai cặp này không thuộc ba cặp gần nhất theo khoảng cách ảnh trung bình, nên khoảng cách centroid chỉ giải thích một phần.
- **MLP:** lỗi phân tán, ba cặp đầu chỉ có 13, 13 và 10 lỗi. Tổng số lỗi của MLP ít hơn Random Forest 237 ảnh (giảm 60,8% số lỗi quan sát được trên cùng tập test).
- **Recall theo lớp thấp nhất:** chữ số 5 ở Naive Bayes (65,8%), Softmax (86,1%) và MLP (97,2%); chữ số 9 ở Random Forest (93,6%). Chữ số 0 và 1 nhận dạng tốt nhất ở hầu hết mô hình.

5.4 Độ chính xác, thời gian và khả năng giải thích

Bảng 5 và Hình 8 đối chiếu accuracy với thời gian huấn luyện và thời gian dự đoán *toàn bộ 10.000 ảnh test*. Thời gian huấn luyện lấy từ lần fit gốc; với MLP, phép đo còn bao gồm đánh giá train/validation sau mỗi epoch (Mục 4).

Thời gian và accuracy:

- Naive Bayes: huấn luyện 0,06 s, dự đoán 0,021 s nhưng kém nhất.
- Softmax Regression: 24,0 s huấn luyện, accuracy 92,22%.



Hình 7. Ma trận nhầm lẫn trên 10.000 ảnh test của bốn mô hình (hàng: nhãn thật; cột: nhãn dự đoán). Dữ liệu train và validation được lưu trong `confusion_matrices.npz`.

Bảng 4. Ba cặp chữ số bị nhầm nhiều nhất trên test theo từng mô hình. $a \to b$ và $b \to a$ là số lỗi theo hai chiều; "Gần nhất" đối chiếu với khoảng cách ảnh trung bình (nguồn: `confusion_pairs.csv`).

Mô hình	Hạng	Cặp	$a \to b$	$b \to a$	Tổng	Gần nhất
Naive Bayes	1	$4 \leftrightarrow 9$	176	67	243	có
	2	$3 \leftrightarrow 5$	18	109	127	không
	3	$5 \leftrightarrow 8$	78	27	105	có
Softmax Regression	1	$4 \leftrightarrow 9$	39	30	69	có
	2	$7 \leftrightarrow 9$	34	27	61	có
	3	$3 \leftrightarrow 5$	21	36	57	không
MLP	1	$3 \leftrightarrow 5$	1	12	13	không
	2	$4 \leftrightarrow 9$	6	7	13	có
	3	$2 \leftrightarrow 7$	5	5	10	không
Random Forest	1	$4 \leftrightarrow 9$	30	15	45	có
	2	$2 \leftrightarrow 7$	10	20	30	không
	3	$3 \leftrightarrow 5$	13	14	27	không

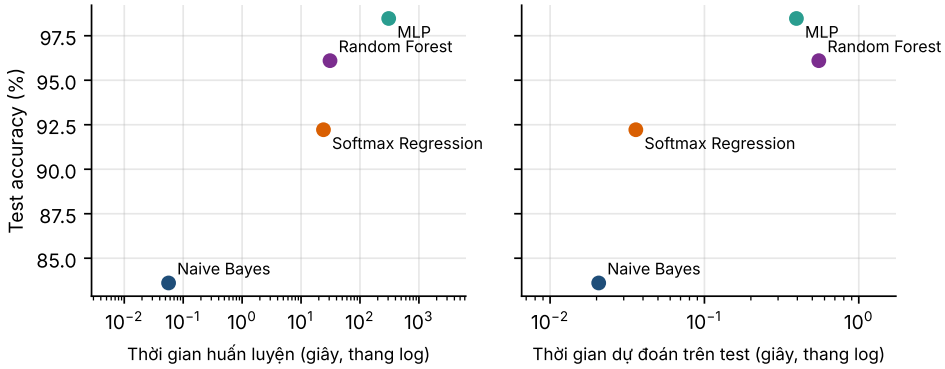
- Random Forest: 31,0 s huấn luyện nhưng dự đoán chậm (0,551 s, vì phải duyệt 50 cây).
- MLP: 307,0 s huấn luyện (gồm đánh giá sau mỗi epoch), dự đoán 0,395 s; đổi lại có accuracy cao nhất.

Khả năng giải thích:

- Naive Bayes và Softmax: các tham số θ_{cj} và trọng số $W_{\cdot k}$ đọc được theo từng pixel của từng lớp, nhưng chỉ phản ánh quan hệ tuyến tính hoặc độc lập có điều kiện.
- Random Forest: cho độ quan trọng đặc trưng theo pixel, tập trung ở vùng giữa ảnh (pixel quan trọng nhất ở hàng 12, cột 14, giá trị 0,0112; 147 pixel có độ quan trọng bằng 0, hệ số tương quan 0,80 với độ lệch chuẩn pixel). Độ quan trọng dựa trên độ giảm tạp chất này có thể thiên lệch và không nêu chiều tác động.
- MLP: không cho ý nghĩa trực tiếp của từng trọng số.

Bảng 5. So sánh cấu hình cuối (nguồn: `final_comparison.csv`, `summary_gap.csv`). Gap = accuracy train − test (pp); huấn luyện: giây/lần fit; dự đoán: giây/10.000 ảnh test.

Mô hình	Acc. test	F1 test	Gap	Huấn luyện	Dự đoán	Quy mô
Naive Bayes	83,61	83,46	-1,18	0,06	0,021	7.850 tham số
Softmax Regression	92,22	92,10	0,19	24,01	0,036	7.850 tham số
MLP	98,47	98,46	1,51	306,96	0,395	814.090 tham số
Random Forest	96,10	96,07	3,57	31,00	0,551	50 cây; 339.142 nút



Hình 8. Accuracy test theo thời gian fit (trái) và thời gian dự đoán toàn bộ test (phải), đều dùng thang log (nguồn: `final_comparison.csv`).

5.5 Đối chiếu với scikit-learn

Bảng 6 đối chiếu bốn cấu hình triển khai của nhóm với các mô hình scikit-learn tương ứng trên cùng dữ liệu; riêng Random Forest sử dụng cây cơ sở của scikit-learn.

- **Naive Bayes:** kết quả trùng hoàn toàn (83,61%) với `MultinomialNB` cùng α.
- **Softmax Regression:** thấp hơn `LogisticRegression` (solver `lbfgs`, $C = 1$ chọn trên validation, `max_iter`= 1000) 0,19 pp trên test. Hai mô hình khác nhau về bộ tối ưu (SGD 20 epoch so với 245 vòng lặp của solver mặc định) và quy ước điều chuẩn ($\lambda = 0$ so với $C = 1$), nên đây không phải so sánh đồng nhất về điều kiện tối ưu. Ở lần chạy đã lưu, mô hình đối chiếu không phát sinh cảnh báo đạt giới hạn `max_iter`.
- **MLP:** bản tự cài đặt cao hơn `MLPClassifier` 0,12 pp. Bản scikit-learn dùng cùng kiến trúc, η, μ, lô và chạy đúng 19 epoch, nhưng không có dropout.
- **Random Forest:** thấp hơn `RandomForestClassifier` 0,12 pp. Cả hai đều dùng cây quyết định của scikit-learn, nhưng khác cách kết hợp (scikit-learn lấy trung bình xác suất lớp, nhóm dùng biểu quyết cứng) và cách triển khai. Việc `fit`/`predict` của thư viện được tối ưu, trong khi bản của nhóm đếm

phiếu bằng vòng lặp Python theo từng mẫu (thư viện: 19,1 s và 0,164 s; nhóm: 31,0 s và 0,551 s). Chưa xác định nguyên nhân chính xác của chênh lệch thời gian.

Các chênh lệch accuracy test trong bảng đều dưới 0,2 pp. Do các mô hình được đánh giá trên cùng tập test và chưa thực hiện kiểm định ghép cặp, không kết luận rằng các chênh lệch này có ý nghĩa thống kê.

Bảng 6. Cấu hình triển khai của nhóm và mô hình scikit-learn tương ứng (accuracy, F1 macro: %; thời gian: giây; nguồn: `sklearn_comparison.csv`). Phần cây cơ sở của Random Forest nhóm dùng scikit-learn.

Cặp	Cài đặt	Acc. val	Acc. test	F1 test	Huấn luyện	Dự đoán
Naive Bayes	Tự cài đặt	82,57	83,61	83,46	0,06	0,021
	scikit-learn	82,57	83,61	83,46	0,29	0,058
Softmax Regression	Tự cài đặt	91,87	92,22	92,10	24,01	0,036
	scikit-learn	92,14	92,41	92,30	80,85	0,041
MLP	Tự cài đặt	98,32	98,47	98,46	306,96	0,395
	scikit-learn	98,26	98,35	98,34	217,35	0,258
Random Forest	Tự cài đặt	95,94	96,10	96,07	31,00	0,551
	scikit-learn	96,17	96,22	96,19	19,15	0,164

6 Thảo luận tổng hợp và giới hạn đánh giá

6.1 Từ khảo sát siêu tham số đến kết quả cuối

Các khảo sát ở Mục 5.2 cho thấy hiệu quả của việc điều chỉnh siêu tham số không đồng đều giữa các họ mô hình.

- **Naive Bayes** chỉ biến thiên 0,36 điểm phần trăm (pp) validation accuracy trong lưới α; do đó thay đổi smoothing khó khắc phục giới hạn của giả định độc lập có điều kiện.
- **Softmax Regression** nhạy với learning rate và điều chuẩn: $\lambda = 10^{-2}$ thu hẹp khoảng cách train–validation nhưng làm validation accuracy giảm còn 89,87%. Điều này minh họa rằng khoảng cách train–validation nhỏ không tự nó bảo đảm chất lượng dự đoán cao.

Với MLP:

- **Learning rate và momentum** tạo ra khác biệt lớn nhất: trong 15 cấu hình thử trong 5 epoch, kết quả validation dao động từ 9,19% đến 97,47%.
- **Kiến trúc:** mạng $[1024]$ đạt 97,56% sau 5 epoch, nhỉnh hơn các kiến trúc được thử trong cùng điều kiện; không thể từ đó kết luận mạng sâu luôn kém hơn.
- **Dropout:** $p = 0{,}2$ trên baseline $[256, 128]$ cải thiện validation accuracy 0,32 pp so với $p = 0$, nhưng đây *không phải* thí nghiệm tách riêng tác động dropout trên MLP cuối $[1024]$.
- **Early stopping** chọn trọng số theo validation loss thay vì accuracy; ở khảo sát riêng, chạy đủ 30 epoch có accuracy cao hơn nhưng validation loss lớn hơn.
- **Cấu hình cuối** 784–1024–10 là kết quả ghép lựa chọn từ các khảo sát có điều kiện khác nhau, rồi huấn luyện lại và chọn epoch tốt nhất; không phải nghiệm tối ưu toàn cục của một lưới đồng thời.

Với Random Forest:

- Rừng 50 cây cải thiện 8,56 pp validation accuracy so với một cây đơn cùng độ sâu tối đa 15.
- Mức tăng từ 30 lên 50 cây chỉ 0,22 pp, trong khi thời gian khảo sát tăng từ 19,6 lên 34,9 s.
- Thí nghiệm ủng hộ lợi ích của tổ hợp cây nhưng chưa xác định điểm bão hòa khi số cây vượt 50.

6.2 So sánh hiệu năng và chi phí theo mục tiêu

Bảng 5 cho thấy không có một mô hình dẫn đầu đồng thời về độ chính xác và tốc độ.

- MLP đạt 98,47% accuracy test, cao hơn Random Forest 2,37 pp nhưng thời gian fit khoảng 307,0 s so với 31,0 s (xấp xỉ 9,9 lần). Đổi lại, trên cùng 10.000 ảnh test, MLP có 153 lỗi so với 390 lỗi của Random Forest.
- Random Forest có lợi thế khi cần một phương án cân bằng giữa accuracy và thời gian huấn luyện trong bốn cấu hình đã đo; không có nghĩa là phương án tối ưu cho mọi môi trường.

Theo từng mô hình:

- **Naive Bayes** nhanh nhất cả khi fit (0,06 s) và dự đoán toàn bộ test (0,021 s), nhưng accuracy chỉ 83,61%.
- **Softmax Regression** đạt 92,22% và cho phép xem trọng số theo pixel/lớp, song biên quyết định tuyến tính hạn chế khả năng biểu diễn.
- **Random Forest** cung cấp độ quan trọng đặc trưng theo pixel nhưng thời gian dự đoán 0,551 s là cao nhất trong bốn cấu hình.
- **MLP** đạt accuracy tốt nhất nhưng khó diễn giải từng trọng số trực tiếp.

Vì vậy, lựa chọn phụ thuộc mục tiêu ưu tiên: độ chính xác, tốc độ huấn luyện, độ trễ dự đoán hay khả năng giải thích (Mục 5.4).

6.3 Khả năng tổng quát hóa và phân tích lỗi

- MLP đạt 99,98% accuracy train nhưng 98,47% trên test (gap 1,51 pp).
- Random Forest đạt 99,67% và 96,10% (gap 3,57 pp).
- Hai mô hình phi tuyến khớp train rất cao, nhưng mức chênh lệch không giống nhau. Kết quả này cần được đọc cùng validation và lịch sử huấn luyện, thay vì quy toàn bộ khác biệt cho overfitting hoặc một kỹ thuật điều chuẩn riêng lẻ.
- Naive Bayes có accuracy test cao hơn train, phản ánh sự khác biệt giữa các tập và không phải bằng chứng về khả năng tổng quát hóa vượt trội.

Phân tích confusion matrix (Mục 5.3) bổ sung thông tin mà accuracy tổng thể không thể hiện:

- cặp $4 \leftrightarrow 9$ đứng đầu ở ba mô hình;
- các lỗi $3 \leftrightarrow 5$ và $2 \leftrightarrow 7$ cho thấy khoảng cách giữa ảnh trung bình của các lớp chỉ giải thích một phần hiện tượng nhầm lẫn;
- MLP giảm đáng kể số lỗi tổng thể nhưng vẫn có những nhầm lẫn ở các cặp nét chữ gần nhau.

Vì vậy, kết luận mô hình tốt hơn cần dựa cả vào độ đo macro và cấu trúc lỗi, không chỉ accuracy.

6.4 Đối chiếu thư viện và hạn chế

Các chênh lệch accuracy test giữa cấu hình nhóm và scikit-learn đều không quá 0,19 pp (Bảng 6), nhưng điều kiện huấn luyện không hoàn toàn tương đương:

- Softmax dùng SGD trong khi LogisticRegression dùng `lbfgs`;
- MLP đối chiếu không dùng dropout;
- Random Forest khác cách tổng hợp dự đoán.

Đây là kiểm tra tính hợp lý của kết quả triển khai, không phải phép chứng minh hai thuật toán hay hai bộ tối ưu tương đương.

Việc sử dụng `DecisionTreeClassifier` làm cây cơ sở cho Random Forest đã được giảng viên chấp thuận và được công bố trong Mục 3.4; không xem đây là hạn chế của đồ án.

Các giới hạn còn lại gồm:

(i) một seed và một lần chia train/validation, chưa đánh giá độ biến thiên qua nhiều lần chạy;
(ii) các lưới khảo sát hữu hạn và điều kiện số epoch khác nhau, đặc biệt đối với MLP;
(iii) thời gian đo trên môi trường Colab CPU dùng chung, với MLP bao gồm đánh giá sau mỗi epoch;
(iv) chỉ sử dụng MNIST với pixel làm phẳng, chưa khảo sát tăng cường dữ liệu hay bộ dữ liệu ngoài miền.

Các chênh lệch nhỏ với scikit-learn chưa được kiểm định ghép cặp nên không suy diễn ý nghĩa thống kê.

Hướng mở rộng. Nếu tiếp tục nghiên cứu, nhóm có thể:

- lặp lại với nhiều seed;
- kiểm định các cặp dự đoán trên cùng tập test;
- khảo sát kết hợp siêu tham số MLP trong cùng ngân sách epoch;
- mở rộng số cây Random Forest để xác định mức lợi ích biên.

Những hướng này không thuộc kết quả đã báo cáo.

7 Kết luận

Nhóm đã xây dựng quy trình huấn luyện, chọn siêu tham số trên validation và đánh giá trên test cho bốn bộ phân lớp MNIST. Các kết quả chính:

- **Độ chính xác:** MLP một lớp ẩn 1024 nơ-ron đạt kết quả tốt nhất ($98{,}47\%$ accuracy, $98{,}46\%$ macro-F1 trên test), tiếp theo là Random Forest ($96{,}10\%$), Softmax Regression ($92{,}22\%$) và Naive Bayes ($83{,}61\%$).
- **Thời gian:** Naive Bayes nhanh nhất ($0{,}06$ s huấn luyện, $0{,}021$ s dự đoán). MLP tốn nhiều thời gian nhất (307 s) nhưng chính xác nhất. Random Forest ở mức trung gian nhưng có khoảng cách train–test lớn ($3{,}57$ pp).
- **Lỗi:** cặp 4–9 là cặp bị nhầm nhiều nhất ở ba trong bốn mô hình.
- **So với scikit-learn:** các cài đặt tự viết khớp ở Naive Bayes và chênh lệch test accuracy không quá $0{,}19$ pp ở ba mô hình còn lại.
- **Hạn chế:** kết quả dựa trên một seed và chưa đánh giá độ biến thiên qua nhiều seed.

Phần cây cơ sở scikit-learn của Random Forest là phương án đã được giảng viên chấp thuận và được công bố trong báo cáo.

Phân công công việc và khai báo sử dụng AI

Phân công và tỷ lệ đóng góp. Cả ba thành viên cùng tham gia phát triển, kiểm tra và hoàn thiện mã nguồn các mô hình trong thư mục `src/`; các nhiệm vụ phụ trách chính được phân công như sau.

Thành viên	MSSV	Nhiệm vụ phụ trách chính	Đóng góp
Lê Huy	24110022	Biên soạn báo cáo; tổng hợp, diễn giải kết quả và cùng phát triển mã nguồn mô hình.	**100%**
Trương Hoàng Phúc	24280020	Xây dựng Notebook, tổ chức và xuất kết quả thí nghiệm; cùng phát triển mã nguồn mô hình.	**100%**
Trần Phương Thảo	24110242	Phụ trách triển khai demo nhận dạng chữ số; cùng phát triển mã nguồn mô hình.	**100%**

Khai báo sử dụng AI. Nhóm có sử dụng công cụ AI để hỗ trợ giải thích kiến thức, gợi ý cấu trúc chương trình, rà soát mã nguồn, hướng dẫn kiểm thử và biên tập báo cáo. Nhóm chịu trách nhiệm kiểm tra, chạy thử, đối chiếu và giải trình toàn bộ mã nguồn, số liệu và nội dung đã nộp.

Mã nguồn, kết quả và khả năng tái lập. Mã nguồn các mô hình, Notebook thực nghiệm và hướng dẫn chạy được quản lý trên GitHub [7]; bảng kết quả và hình ảnh thực nghiệm được lưu trữ tại [6] và [5].

GitHub | Results | Figures

Các tệp kết quả và hình ảnh thực nghiệm được lưu trữ trong các thư mục `results/` và `figures/` của bộ kết quả Colab.

Tài liệu tham khảo

[1] Fabian Pedregosa et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.

[2] BTVN-Lab: Bộ bài thực hành học phần Nhận dạng. Học liệu thực hành, Google Drive. URL https://drive.google.com/drive/folders/1FZH8b6k4XHNkpL4tGAITW0_RWYzwieWV. Tuần 1–5; truy cập 07/11/2026.

[3] K. Wickström. Pattern recognition handbook. Sổ tay bài giảng FYS-3012 Pattern Recognition, UiT The Arctic University of Norway; GitHub, MIT License, n.d. URL https://github.com/Wickstrom/pattern-recognition-handbook. Tài liệu được dẫn trong BTVN-Lab Tuần 3 và Tuần 4.

[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] Nhóm thực hiện đồ án MNIST. Hình ảnh và biểu đồ thực nghiệm MNIST (thư mục figures). Google Drive, 2026. URL https://drive.google.com/drive/folders/1Lo55d6jmuT2Pg9kPYbrCsZW4vr8mclml. Truy cập 07/11/2026.

[6] Nhóm thực hiện đồ án MNIST. Kết quả thực nghiệm MNIST (thư mục results): bảng số liệu CSV, JSON, NPZ. Google Drive, 2026. URL https://drive.google.com/drive/folders/1Z46gMqCoBIAX-HzQVAtT5JFGVxzu7_Qj. Truy cập 07/11/2026.

[7] Nhóm thực hiện đồ án MNIST. MTH10322-PR-Midterm-Project: mã nguồn, Notebook và hướng dẫn tái lập. GitHub, 2026. URL https://github.com/heleme69/MTH10322-PR-Midterm-Project. Truy cập 07/11/2026.